SPRING 2026

CS 315

PROJECT 2

Instructor: H. Altay Güvenir

TEAM

Language Name: CeroOne

Ata Eren Kılıç	22202801
Elif Tuğçe Neziroğlu	22203839
Mehmet Batuhan Neziroğlu	22203840

BNF Description of CeroOne:

<program> ::= <top_list>

<top_list> ::= <top_item> | <top_item> <top_list>

<top_item> ::= <func_def> | <stmt>

<func_def> ::= func <ret_type> <identifier> (<param_list>) <stmt_block>
| func <ret_type> <identifier> ( ) <stmt_block>

<ret_type> ::= void | <type>

<return_stmt> ::= return ; | return <expr> ;

<param_list> ::= <param> | <param> , <param_list>

<param> ::= <type> <identifier>

<stmt_block> ::= { <stmt_list> } | { }

<stmt_list> ::= <stmt> | <stmt> <stmt_list>

<stmt> ::= <declaration_stmt> | <assign_stmt> | <if_stmt> | <switch_stmt> |
<loop_stmt> | <io_stmt> | <return_stmt> | <func_call_stmt> | <comment>

<declaration_stmt> ::= <type> <decl_list> ;

<decl_list> ::= <declarator> | <declarator> , <decl_list>

<declarator> ::= <identifier> | <identifier> = <init_expr>

<init_expr> ::= <expr>

<type> ::= bool | <type>*

<ident_list> ::= <identifier> | <identifier> , <ident_list>

<assign_stmt> ::= <identifier> = <expr> ;

<func_call_stmt> ::= <identifier> (<arg_list_opt>) ;

<if_stmt> ::= if (<expr>) <stmt_block>
| if (<expr>) <stmt_block> else <stmt_block>

```
| if ( <expr> ) <stmt_block> <elif_chain>
| if ( <expr> ) <stmt_block> <elif_chain> else <stmt_block>

<elif_chain>        ::= <elif_stmt> | <elif_stmt> <elif_chain>

<elif_stmt> ::= elif ( <expr> ) <stmt_block>

<switch_stmt> ::= switch ( <expr> ) { <case_list> <default_opt> }

<case_list>     ::= <case_clause> | <case_clause> <case_list>

<case_clause> ::= case <expr> : <stmt_block>

<default_opt>   ::= default : <stmt_block> | <empty>


<loop_stmt>         ::= <while_stmt> | <for_stmt>

<while_stmt> ::= while ( <expr> ) <stmt_block>

<for_stmt> ::= for <identifier> in <expr> <stmt_block>


<io_stmt>           ::= <input_stmt> | <output_stmt>

<input_stmt>        ::= input ( <ident_list> ) ;

<output_stmt>       ::= print ( <print_list> ) ;

<print_list> ::= <print_arg> | <print_arg> , <print_list>

<print_arg>         ::= <string_literal> | <expr>

<expr>              ::= <biconditional_expr>

<biconditional_expr>     ::= <implication_expr>
                         | <biconditional_expr> <=> <implication_expr>

<implication_expr>       ::= <equality_expr>
                         | <equality_expr> => <implication_expr>

<equality_expr>          ::= <or_expr>
                         | <or_expr> <eq_op> <or_expr>
```

```
<eq_op>              ::= == | !=

<or_expr>            ::= <xor_expr>
                   | <or_expr> || <xor_expr>

<xor_expr>           ::= <and_expr>
                   | <xor_expr> xor <and_expr>

<and_expr>           ::= <not_expr>
                   | <and_expr> && <not_expr>

<not_expr>           ::= ! <not_expr>
                   | <primary_expr>

<primary_expr> ::= <bool_constant>
| nil
| [ ]
| [ <expr_list> ]
| cons ( <expr> , <expr> )
| tail ( <expr> )
| head ( <expr> )
| isEmpty ( <expr> )
| ( <expr> )
| <identifier> <call_opt>

<call_opt> ::= ( <arg_list_opt> )
| <empty>

<arg_list_opt> ::= <arg_list>
| <empty>

<arg_list> ::= <expr>
| <expr> , <arg_list>

<expr_list> ::= <expr>
| <expr> , <expr_list>


<bool_constant>     ::= true | false

<string_literal>    ::= " <string_content> "

<string_content>    ::= <empty> | <char> <string_content>
```

<char> ::= <letter> | <digit> | <special_char>

<identifier> ::= <letter> | <identifier> <id_char>

<id_char> ::= <letter> | <digit> | _ | $

<letter> ::= a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z

<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<special_char> ::= ' ' | '!' | '#' | '$' | '%' | '&' | '(' | ')' | '*' | '+' | ',' | '-' | '.' | '/' | ':' | ';' | '<' | '=' | '>' | '?' | '@' | '[' | '\' | ']' | '^' | '_' | '{' | '|' | '}' | '~' | '\t' | '\n'

<empty> ::=

<comment> ::= <single_line_comment>
| <multi_line_comment>

<single_line_comment> ::= // <line_content>

<multi_line_comment> ::= /* <multi_line_content> */

<multi_line_content> ::= <char_except_star> <multi_line_content>
| * <not_slash> <multi_line_content>
| <empty>

<line_content> ::= <empty> | <line_char> <line_content>

<line_char> ::= <letter> | <digit> | <special_char_no_newline>

<char_except_star> ::= <letter>
| <digit>
| <special_char_no_star>

<not_slash> ::= <letter>
| <digit>
| <special_char_no_slash_no_star>
| * <not_slash>

<special_char_no_star> ::= ' ' | '!' | '#' | '$' | '%' | '&' | '(' | ')' | '+'
| ',' | '-' | '.' | '/' | ':' | ';' | '<' | '='
| '>' | '?' | '@' | '[' | '\' | ']' | '^' | '_'
| '{' | '|' | '}' | '~' | '\t' | '\n'

```
<special_char_no_slash_no_star> ::= ' ' | '!' | '#' | '$' | '%' | '&' | '(' | ')'
                                  | '+' | ',' | '-' | '.' | ':' | ';' | '<' | '='
                                  | '>' | '?' | '@' | '[' | '\' | ']' | '^' | '_'
                                  | '{' | '|' | '}' | '~' | '\t' | '\n'
<special_char_no_newline> ::= ' ' | '!' | '#' | '$' | '%' | '&' | '(' | ')' | '*'
                                  | '+' | ',' | '-' | '.' | '/' | ':' | ';' | '<' | '='
                                  | '>' | '?' | '@' | '[' | '\' | ']' | '^' | '_'
                                  | '{' | '|' | '}' | '~' | '\t'
```

Description of Non-Terminals:

<program>

Represents the complete source file. A program may optionally begin with one or more function definitions followed by a list of executable statements, or it may consist only of statements. This structure allows helper functions to be declared before they are used. Execution conceptually begins from the top-level statement list.

<top_list>

Represents the complete sequence of top-level constructs in a CeroOne program. It consists of one or more <top_item> elements and allows functions and standalone statements to appear at the program level in sequence.

<top_item>

Represents a single top-level construct in a CeroOne program. A top-level item may be either a function definition (<func_def>) or a statement (<stmt>). This allows the program body to contain both function declarations and executable statements.

<func_def>

<func_def> defines a function. A function begins with the keyword func, followed by a return type (<ret_type>), a function name (<identifier>), an optional parameter list, and a statement block (<stmt_block>).

<ret_type>

Specifies the return type of a function. A function may either return void or any valid <type>. This allows both procedures with no return value and value-returning functions.

<param_list>

Represents one or more parameters separated by commas. It cannot be empty unless the function explicitly uses empty parentheses.

<param>

Defines a single function parameter consisting of a <type> followed by an <identifier>. Parameters are passed by value.

<stmt_block>

Represents a compound statement enclosed in {}. It may contain zero or more statements. Curly braces are mandatory to eliminate ambiguity in nested conditionals and loops.

<stmt_list>

Represents one or more consecutive statements. The recursive definition allows programs of arbitrary length.

<stmt>

Defines all valid executable statements in the language:

- Variable declaration
- Assignment
- Conditional statement
- Loop

- Input/output
- Return statement
- Function call
- Comment

Each statement type follows its own syntactic rules.

<declaration_stmt>

Declares one or more variables of a given type. Declarations must end with a semicolon. Variables must be declared before they are used. Multiple identifiers may be declared in a single statement using commas.

<decl_list>

Represents one or more declarators separated by commas inside a declaration statement.

<declarator>

Defines a single declared variable. It may be just an identifier or an identifier initialized with an expression.

<init_expr>

Represents the expression used to initialize a variable at the point of declaration.

<type>

specifies the type of a variable or parameter. The language supports:

- bool (single Boolean value)
- bool* (list of bool)
- bool** (list of list of bool)
- In general: bool followed by zero or more * characters, where each * adds one list dimension.

<ident_list>

Represents a comma-separated list of identifiers used in variable declarations.

<assign_stmt>

Defines an assignment statement. It assigns the value of an <expr> to a single identifier. Assignments must end with a semicolon. Only single-target assignment is allowed.

<return_stmt>

Non-void functions must return a value on all execution paths; void functions may use return; or omit a return statement.

<func_call_stmt>

Represents a function call used as a standalone statement. It must end with a semicolon.

<if_stmt>

Defines conditional execution. Supports:

- if
- if-else
- if-elif
- if-elif-else

Conditions must be enclosed in parentheses. Each branch must use a statement block {}.

<elif_chain>

Represents one or more elif branches following an if statement.

<switch_stmt>

Represents a multi-branch conditional statement that selects one execution path based on the value of a Boolean expression. The switch statement evaluates a controlling <expr> once and compares its result against the Boolean expressions specified in each case clause.

<elif_stmt>

Defines a single elif clause with a <expr> condition and a statement block.

<loop_stmt>

Represents looping constructs. It may be either a while loop or a for loop.

<while_stmt>

Executes a statement block repeatedly as long as the Boolean condition evaluates to true. The condition must be enclosed in parentheses.

<for_stmt>

Defines iteration using the form for <identifier> in <expr> <stmt_block>. The loop variable successively takes values from the expression being iterated. The loop body is enclosed in {}

<io_stmt>

Represents input or output operations.

<call_opt>

Represents the optional function-call suffix following an identifier. An identifier may either stand alone or be followed by parentheses containing optional arguments.

<input_stmt>

Reads values from input and stores them into the listed identifiers. Each identifier receives a value matching its type (bool or bool list). List input must be provided in bracket form, e.g., [true, false, true]. Input must target declared variables.

<output_stmt>

Prints one or more arguments to the console. Arguments may be string literals or expressions.

<print_list>

Represents a comma-separated list of print arguments.

<print_arg>

Represents a single output argument. It may be a string literal or an <expr>.

<expr>

Represents the highest-level general expression construct in the language. It begins at the biconditional precedence level and covers all valid expressions.

<biconditional_expr>

Represents double implication (<=>). It has the lowest precedence among logical operators.

<implication_expr>

Represents implication (=>). It has higher precedence than biconditional but lower than OR.

<or_expr>

Represents logical OR (||). It has lower precedence than AND.

<equality_expr>

Represents equality comparison expressions. It evaluates two logical subexpressions using either == (equality) or != (inequality).
It has lower precedence than || and higher precedence than implication (=>).
Only Boolean operands are allowed.

<eq_op>

Represents equality operators:

- == → Logical equality (true if both operands have the same truth value).
- != → Logical inequality (true if operands differ).

<xor_expr>

Represents logical exclusive OR expressions using the xor operator.
It evaluates to true when exactly one operand is true.
Its precedence is:

- Lower than &&
- Higher than ||

This placement ensures predictable parsing of compound logical expressions.

<and_expr>

Represents logical AND (&&). It has higher precedence than OR but lower than NOT.

<not_expr>

Represents logical negation (!). It is a unary operator with high precedence.

<primary_expr>

Represents the most basic expression forms in the language. These include Boolean constants, nil, empty and non-empty list literals, list operations such as cons, tail, head, and

isEmpty, parenthesized expressions, and identifiers optionally followed by a function-call suffix. Parentheses override normal precedence rules.

<arg_list>

Represents a comma-separated list of arguments.

<arg_list_opt>

Represents an optional argument list in a function call. It may contain arguments or be empty.

<expr_list>

Represents one or more expressions separated by commas. It is used inside list literals.

<bool_constant>

Represents the reserved constants true and false.

<string_literal>

Represents characters enclosed within double quotation marks. Strings are used only in output operations.

<string_content>

Represents zero or more characters inside a string literal.

<char>

Represents a single character allowed inside a string.

<identifier>

Defines variable and function names. Identifiers must begin with a letter. Subsequent characters may be letters, digits, underscore _, or dollar sign $. Identifiers are case-sensitive.

<id_char>

Defines valid characters that may follow the first letter in an identifier.

<letter>

Represents any uppercase or lowercase English alphabet character.

<digit>

Represents numeric characters 0 through 9.

<special_char>

Defines the allowed non-alphanumeric characters for strings and comments.

<empty>

Represents the empty string (ε). Used for optional constructs.

<comment>

Represents comments in the language. Comments are ignored during execution.

<single_line_comment>

Begins with // and continues until the end of the line.

<multi_line_comment>

Begins with /* and ends with */.

<multi_line_content>

Defines valid content inside multi-line comments while preventing premature termination.

<char_except_star>

Defines characters allowed inside multi-line comments except *, preventing accidental closing.

<line_content>
Represents the content of a single-line comment after // until the end of the line.

<line_char>
Represents a single character allowed inside a single-line comment.

<special_char_no_newline>
Defines the set of special characters allowed in a single-line comment, excluding newline.

<not_slash>

Defines characters that may follow * inside multi-line comments, but prevents forming */ unintentionally.

<special_char_no_star>

Defines special characters allowed inside comments except *.

<special_char_no_slash_no_star>

Defines special characters allowed after * in multi-line comments, except / and *.

Terminals

!= : not equals operator
== : equality operator
= : assignment operator

&& : logical AND operator
|| : logical OR operator
xor : logical exclusive OR operator
! : logical NOT operator
=> : logical implication operator
<=> : logical biconditional operator

(: left parenthesis
) : right parenthesis
{ : left curly bracket
} : right curly bracket
[: left square bracket
] : right square bracket

, : comma
; : semicolon
: : colon

// : single-line comment indicator
/* : multi-line comment start
*/ : multi-line comment end

func

Used to define a function. Every function definition must begin with this keyword. It clearly distinguishes function declarations from other statements.

bool

Represents the primitive Boolean type. It is used in variable, parameter, and function return type declarations.

*****

Used in type formation. Each * applied to a type forms a higher-level aggregate type recursively; for example, bool* is a list of Boolean values and bool** is a list of Boolean lists.

return

return exits a function. In non-void functions, it returns a value (bool or bool list). In void functions it may appear as return; (or be omitted, if allowed).

if

Introduces a conditional statement. It must be followed by a parenthesized Boolean expression and a statement block.

elif

Represents an additional conditional branch evaluated only if previous conditions are false.

else

Defines the default branch of a conditional statement when no previous condition evaluates to true.

switch

Introduces a multi-branch conditional structure. It evaluates a Boolean expression and selects a matching case block to execute. The statement body must be enclosed in curly braces.

case

Defines a conditional branch inside a switch statement. It specifies a Boolean expression and an associated statement block that executes if the expression matches the switch condition.

default

Defines the fallback branch of a switch statement. It executes when none of the case conditions evaluate to true.

while

Introduces a loop that repeatedly executes a block as long as its Boolean condition evaluates to true.

for

Introduces an iteration over a Boolean list using the for identifier in list structure.

in

Used within for loops to separate the loop variable from the list expression being iterated.

input

Reads values from input and stores them into the listed identifiers. Each identifier receives a value matching its type (bool or bool list). List input must be provided in bracket form, e.g., [true, false, true].

print

Represents console output. It prints string literals and/or Boolean expressions.

true

Boolean constant representing logical truth.

false

Boolean constant representing logical falsehood.

nil

Represents the empty Boolean list.

cons

A list-construction operator that adds a Boolean element to the front of a list.

head

Returns the first element of a Boolean list.

tail

Returns the list obtained by removing the first element.

isEmpty

Returns a Boolean value indicating whether a list is empty.

Evaluation of the Language

Readability

The language is designed to be structurally clear and logically focused. Since it operates only on Boolean values and Boolean lists, programs remain conceptually simple and easy to follow. Restricting data types eliminates unnecessary complexity and reduces cognitive load when reading code.

Mandatory curly braces for all statement blocks remove ambiguity, especially in nested conditional and loop structures. This prevents the classical “dangling else” problem and ensures that block boundaries are always visually clear.

The operator hierarchy is explicitly defined in the grammar, ensuring predictable interpretation of logical expressions. Using symbolic logical operators (!, &&, ||, =>, <=>) keeps expressions concise while maintaining familiarity with standard logical notation.

Reserved keywords such as func, print, input, isEmpty, head, and tail are chosen to be descriptive. Their meaning is directly inferable, which improves clarity and reduces misunderstanding.

Restricting strings to output operations also improves readability by clearly separating logical computation from display behavior.

Writability

The language is intentionally minimal, which improves the ease of writing correct programs. Because only Boolean and Boolean list types exist, the programmer does not need to manage multiple data types or complex type conversions.

List operations such as cons, head, tail, and isEmpty provide expressive power while remaining simple. They allow structured manipulation of Boolean collections without introducing complicated data structures.

The for identifier in list construct simplifies iteration over Boolean lists. This avoids manual index management and makes list traversal straightforward.

Symbolic logical operators reduce verbosity in expressions. For example, complex logical statements can be written compactly using => and <=>.

The strict structure of function definitions (supporting bool, bool list, and void return types) and consistent statement blocks make the language easy to write and reason about.

Reliability

The language enforces strong structural and typing constraints, which increases reliability. Variables must be declared before use, and the allowed types are bool and bool list. This prevents unintended type mixing.

Functions are restricted to returning values that match their declared return type: bool, bool list, or void. This prevents type mismatches in returns and improves reliability.

Reserved keywords cannot be used as identifiers, preventing naming conflicts. Identifiers must begin with letters, which eliminates lexical ambiguity with numeric or symbolic tokens.

Operator precedence is clearly defined through layered non-terminals in the grammar. This guarantees deterministic parsing of logical expressions.

Mandatory semicolons and block delimiters further reduce syntactic ambiguity. Comments are well-defined to prevent accidental premature termination, especially in multi-line comments.

By limiting the language to Boolean logic and aggregate Boolean lists, the overall design reduces implementation complexity and minimizes sources of semantic errors.

Beginning of Execution

A program consists of zero or more function definitions followed by a top-level statement list. Execution begins at the first statement in this top-level <stmt_list> and proceeds sequentially in source order. Function definitions are parsed and made available before execution starts, but they are not executed unless called. There is no special required entry function (such as main); the “entry point” is simply the first executable statement at the top level. For further reference, please look at the beginning of the BNF.

Declarations

Declarations introduce new identifiers and assign them a type within the current scope. In this language, declarations are statements, so they may appear wherever statements are allowed: at the top level or inside any { ... } block. A declaration can be written without initialization (T x;) or with initialization at the point of declaration (T x = <init_expr>;). The initializer

may be either a boolean expression or a list expression, and it must type-check to the declared type T. List types are formed using *, meaning “list of”: for example, bool* is a list of booleans and bool** is a list of lists, allowing multidimensional lists. Scoping is block-based: a variable is visible from its declaration to the end of the block in which it appears. Redeclaring the same identifier within the same scope is not allowed.

Higher to Lower Operator Precedence

1. Parentheses: (...)
2. Logical NOT: !
3. Logical AND: &&
4. Logical XOR: xor
5. Logical OR: ||
6. Equality / Not-equals: ==, !=
7. Implication: =>
8. Biconditional: <=>

Notes:

- => is right-associative; the others are left-associative.
- = is used for assignment statements, not as part of boolean-expression precedence.
- Precedence of logical operators are covered in BNF as <bool_expr> is expended from <biconditional_expr> to <not_expr>

Evaluation order:

In all expressions, operands are evaluated left-to-right. The operators && and || use short-circuit evaluation (the right operand is evaluated only if needed). All other binary operators (xor, ==, !=, =>, <=>) evaluate both operands.

Operator Associativity: Most logical operators in the language, including &&, xor, ||, ==, and !=, are left-associative. This means that in an expression like A || B || C, the operation is grouped as (A || B) || C.

Right-Associativity for Implication: The implication operator => is right-associative. Therefore, A => B => C is evaluated as A => (B => C), following standard mathematical logic conventions.

Parameter passing method

All function parameters are passed by value. When a function is called, actual argument expressions are evaluated left-to-right, and the resulting values are copied into the corresponding formal parameters. This provides clear, deterministic behavior and prevents accidental aliasing. For list values, the semantics are still value-based: operations such as cons, head, tail, and list literals produce or inspect list values without mutating existing lists, so passing a list into a function cannot change the caller’s list. An implementation may internally share list structures for efficiency, but this optimization must not change the program's observable behavior. In other words, a function must not be able to modify the caller’s list through a parameter.